

Documento de diseño técnico

Autor: Andres Felipe Blanco Centeno

Fecha: 2026-09-04

Resumen

Este documento describe la arquitectura, el diseño de módulos y el modelo de datos recomendado para el proyecto **prueba-AI** (asistente RAG local para una academia de idiomas). Su objetivo es servir como evidencia técnica y guía para implementación, empaquetado y entrega.

Alcance

- Registrar cómo está construido el sistema y justificar decisiones técnicas.
- Definir el modelo de datos relacional recomendado y proveer scripts SQL de ejemplo.
- Señalar archivos y lugares donde deben colocarse los artefactos solicitados en la evidencia (diagramas, mockups, manuales).

Arquitectura general

El sistema tiene dos capas principales:

- Frontend: Vue 3 + Vite + Tailwind CSS. Se ejecuta en **frontend/** y comunica con el backend mediante la ruta relativa **/api/*** (Vite proxy).
- Backend: FastAPI que orquesta la indexación RAG, habilidades deterministas y la interacción con Ollama. Código en **backend/**.

Componentes clave

- **backend/main.py** — API, CORS, healthcheck, orquestación RAG y manejo de inscripciones.
- **backend/rag_engine.py** — carga de documentos (**backend/analizar/**), fragmentación y obtención de embeddings mediante Ollama.
- **backend/ollama_client.py** — genera prompts y consulta el endpoint de Ollama.
- **backend/skills.py** — lógica determinista para inscripción y conversiones.
- Frontend en **frontend/src/** — componentes Vue que gestionan la sesión de chat y el envío de **session_id**.

Flujo RAG

1. Los documentos fuentes (**.md/.txt**) se colocan en **backend/analizar/**.
2. Al iniciar, **RAGEngine.load_documents()** fragmenta y solicita embeddings a Ollama.
3. En la consulta **/chat**, el backend obtiene un embedding para la consulta y recupera los chunks más relevantes.
4. El prompt final (contexto + instrucciones de sistema) se envía a Ollama para generar la respuesta.

Justificación técnica

- Uso de Ollama: permite mantener todo el pipeline local para privacidad y reproducibilidad.
- **nomic-embed-text** para embeddings: coherente con el cliente Ollama y fácil de integrar.
- FastAPI: ligero y fácil de desarrollar APIs REST en entornos locales.

Persistencia y migración a SQL

Actualmente las inscripciones se guardan en **backend/student_registrations.json**. Para un entorno de producción o evaluación formal, recomendamos migrar a una base de datos relacional. Propuesta:

- Tabla `students` con datos principales del alumno.
- Tabla `courses` con idiomas ofrecidos.
- Tabla `registrations` que referencia `students` y `courses`.

El diagrama ER y el script SQL de ejemplo se encuentran en: - docs/erd.md - db/schema.sql

Requisitos de evidencia y entregables

Para cumplir con las imágenes entregadas (lista de evidencias), almacene los artefactos siguientes en el repositorio antes de crear el ZIP:

- docs/diagrams/uml-*.png o docs/diagrams/uml-*.drawio — diagramas de casos de uso, clases y secuencia.
- docs/mockups/* — wireframes o capturas de las pantallas principales.
- docs/design_document.md — este documento.
- db/schema.sql — scripts de creación de tablas.
- backend/backend.md y frontend/frontend.md — manuales/instructivos (ya presentes).

Pruebas y verificación

1. Verificar endpoints: GET /health y POST /chat.
2. Confirmar que Ollama responde a `ollama list` y que los modelos `nomic-embed-text` y `llama3` (o `MODEL_NAME`) están disponibles.
3. Revisar que `backend/analizar/` contenga al menos 3 documentos de negocio (requisito actual del inicializador).

Tareas siguientes (recomendadas)

- Añadir diagramas UML en docs/diagrams/ (casos de uso, clases, secuencia/actividad).
- Exportar mockups y guardarlos en docs/mockups/.
- Decidir si migrar `student_registrations.json` a una BD relacional y ejecutar `db/schema.sql`.

Contacto

Si quieres que genere automáticamente los diagramas iniciales (ERD en PNG) o que aplique la migración SQL a una base de datos local para pruebas, indícamelo y procedo.